Aurela's Section: Senior Design I Documentation Early Status Report

As of right now, I began working with the AlgoTune benchmark suite inside OpenEvolve. Cloned the repository and installed all required Python dependencies inside the virtual environment. Created and activated the virtual environment to keep the dependencies contained. Did unit testing and baseline runs to verify the installation. I successfully ran initial tasks such as function˙minimization and produced results.

My current effort is focused on preparing and running AlgoTune tasks such as polynomial˙real, convolve2d˙full˙fill, and working on testing attention˙optimization to evaluate performance improvements.

Right now, my focus is on setting up and running OpenEvolve experiments within the AlgoTune benchmark suite. I cloned the repository, created and activated a virtual environment, and installed all required dependencies. I verified the installation by running unit tests and baseline examples. I still need to test other examples (e.g., robust regression in R, symbolic regression on LLM-SRBench) and document both the environment setup steps and how the model behaves during evolution.

My work so far has been directed toward understanding how OpenEvolve evolves algorithmic solutions across tasks, starting with smaller examples like function_minimization and moving toward more complex AlgoTune optimizations.

Beyond benchmark tasks, I also discovered that OpenEvolve can solve competitive programming problems such as those hosted on Kattis. For instance, in the *Alphabet* problem, the system started from a incorrect solution and, after a few iterations, evolved into a fully correct dynamic programming approach. This demonstrates OpenEvolve's ability to discover well-known algorithms such as the Longest Increasing Subsequence (LIS), and validates its generality beyond just optimization benchmarks.

Evolved Algorithm Example (Kattis: Alphabet)

```python
import sys

for line in sys.stdin:
    s = line.strip()

n = len(s)
dp = [1] * n

for i in range(1, n):
    for j in range(i):
        if s[i] > s[j]:
            dp[i] = max(dp[i], dp[j] + 1)

longest_alphabetical_subsequence_length = max(dp)
ans = 26 - longest_alphabetical_subsequence_length
print(ans)
```

This finding highlights that OpenEvolve can use online judge platforms as a "fitness function" to evolve correct and efficient algorithms from scratch. The workflow uses Kattis's submission pipeline, where a helper script (`submit.py`) automates the process of sending source code to the Kattis judge. This script reads the command line arguments (solution file names), infers the problem ID and language from extensions (e.g., `.py` → Python 3, `.cpp` → C++, `.f90` → Fortran), logs in using the local `.kattisrc` credentials, and posts the submission via HTTP. The server then compiles and runs the solution against **hidden test cases** , while the script continuously polls the results and displays progress back in the terminal. In effect, Kattis itself acts as the evaluation harness while OpenEvolve drives the code mutations until a passing solution emerges.

In addition to online judge testing, OpenEvolve is able to improve symbolic regression models by **evolving actual Python code** that contains explicit mathematical formulas. This means that you can open the `initial_program.py` file at any generation and directly observe how the code changes as the algorithm improves. For example:
Initial generation:

```python
def func(x, params):
    return params[0]*x[:,0] + params[1]*x[:,1]
```

After a few generations:

```python
def func(x, params):
    return -params[0]*x[:,0] - params[1]*x[:,0]**3 + params[2]*np.sin(x[:,1])
```

Later generation:

```python
def func(x, params):
    return -params[0]*x[:,0] - params[1]*x[:,0]**3 - params[2]*x[:,0]*x[:,1] +
    ↪  params[3]*np.sin(x[:,1])
```

This makes the evolutionary process transparent, since you can watch a naive linear model transform into a nonlinear expression with polynomial, interaction, and trigonometric terms that better capture the dataset.

This complements my current work with AlgoTune benchmarks and strengthens the foundation for using evolutionary approaches in both performance optimization and general algorithm discovery.

Evolution Summary

```
Evolution complete!
Best program metrics:
  runs_successfully: 1.0000
  value_score:       0.9418
  distance_score:    0.7566
  combined_score:    1.2148
```

Checkpoint Information

```
Latest checkpoint saved at:
examples/function_minimization/openevolve_output/checkpoints/checkpoint_100

To resume, use:
--checkpoint examples/function_minimization/openevolve_output/checkpoints/checkpoint_100
```